CRYPTOGRAPHY
Analysis Report

Fundamentals of Cryptography

Symmetric vs Asymmetric Systems

- Author -
Abdessamad Ait Brahim

- Professor -
Prof. A. HARCHAOUI

- Course -
Security M232 & Cryptography

December 7, 2025

1 Symmetric Cryptography

1.1 Advanced Encryption Standard

The AES encryption algorithm appeared after DES failed. Below is the architecture of AES. This image will help to understand it step by step.

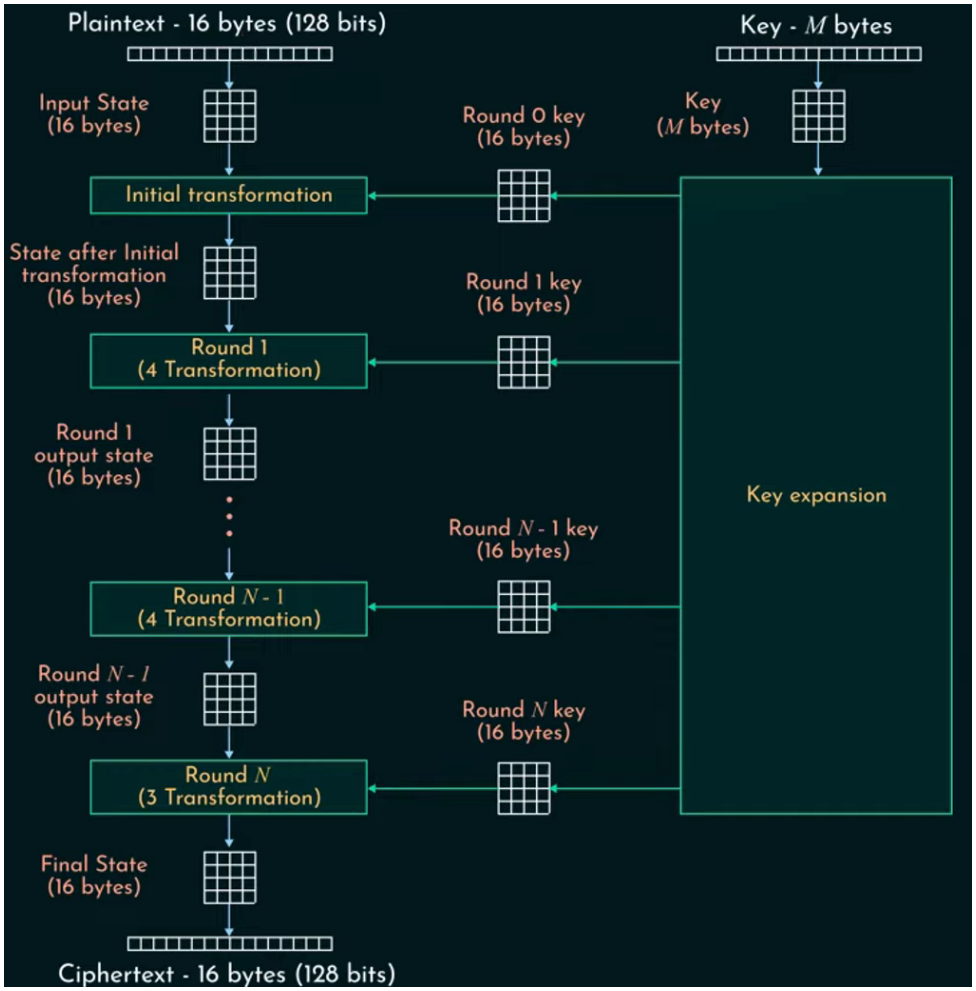


Figure 1: General architecture of AES

The AES algorithm operates based on several key principles:

- **Cipher Type:** AES is a **block cipher** that processes data in fixed blocks of **16 bytes (128 bits)**.
- **Key Expansion:** The secret **Key** undergoes a **Key Expansion** process to generate all the unique **Round Keys** required for the entire encryption.
- **Iterative Process:** AES operates through repeated steps called **Rounds**. Each key size defines a specific number of rounds and the amount of round keys generated:
  - **AES-128:** 10 rounds, produces 11 round keys.
  - **AES-192:** 12 rounds, produces 13 round keys.

  - **AES-256:** 14 rounds, produces 15 round keys.

- **Initial Transformation:** The process begins with the first **AddRoundKey** operation (Initial Transformation) combining the Plaintext with the Round 0 Key.
- **Round Structure:** Full rounds consist some core transformations The final round is slightly modified.
- **Output:** After the final round, the resulting **Final State** is produced as the **Ciphertext**.

1.2 AES Encryption and Decryption

This illustration helps us clearly understand what happens during AES processing and allows us to drill down into each round transformation.

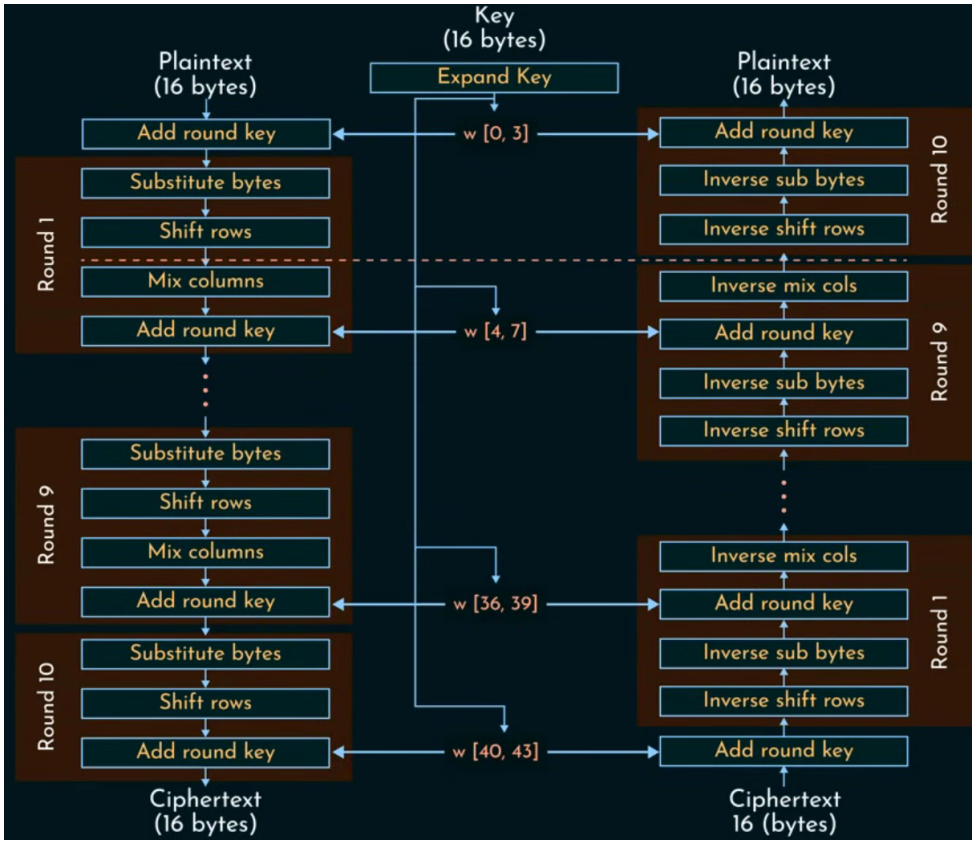


Figure 2: AES Encryption and Decryption

- **Encryption (left side):**
  - Starts with an initial **AddRoundKey**.
  - Rounds 1 to 9 apply: **SubBytes**, **ShiftRows**, **MixColumns**, and **AddRoundKey**.
  - The final round removes MixColumns and performs only: **SubBytes**, **ShiftRows**, and **AddRoundKey**.
  - Produces the final **Ciphertext**.

- **Decryption (right side):**
  - Starts from the ciphertext and applies the inverse operations: **InverseSubBytes**, **InverseShiftRows**, **InverseMixColumns**, and **AddRoundKey**.
  - Rounds are executed in reverse order: from Round 10 down to Round 1.
  - Restores the original **Plaintext**.
- **Key Expansion:**
  - The 128-bit key is expanded into 44 words: `w[0] ... w[43]`.
  - Every 4 words form one round key used during both encryption and decryption.

1.3 AES Transformation Functions

Each AES round applies four main transformations to the state. The first two transformations of each round are **SubBytes** and **ShiftRows**. The following figure illustrates both steps:

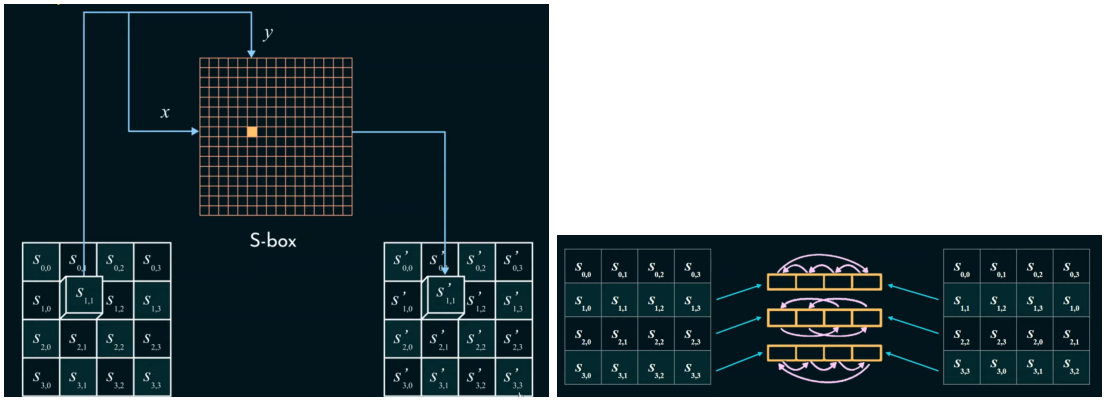


Figure 3: AES first transformations: SubBytes (left) and ShiftRows (right).

SubBytes

Each byte of the state is replaced using the AES S-box, a non-linear substitution table designed to protect against linear and differential cryptanalysis. This step introduces strong *confusion*: even a small change in the input byte produces a completely different output byte.

ShiftRows

The rows of the state matrix are cyclically shifted to the left by different offsets (0, 1, 2, and 3 bytes). This operation spreads the bytes across columns, increasing *diffusion* and ensuring that the effect of a single substituted byte propagates through the entire block in later rounds.

The next two transformations are MixColumns and AddRoundKey:

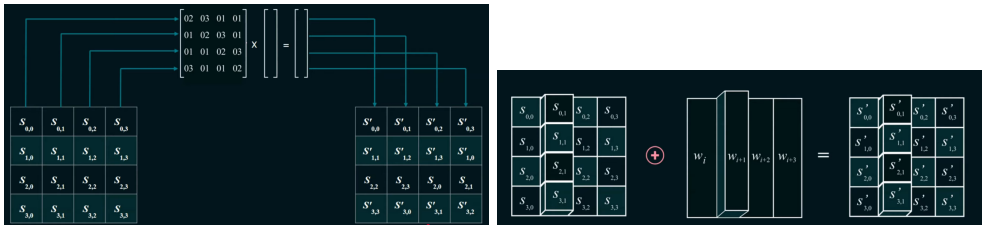


Figure 4: AES next transformations: MixColumns (left) and AddRoundKey (right).

MixColumns

In this step, each column of the state matrix is treated as a four-term polynomial and multiplied by a fixed matrix in *GF(2^8)*. This operation provides strong *diffusion*, spreading the influence of each byte across the column so that a change in one byte affects all four bytes in the column.

AddRoundKey

Each byte of the state matrix is XORed with the corresponding byte of the round key for the current round. This step introduces the secret key into the encryption process, providing *confusion* and making the ciphertext dependent on the key.

1.4 AES Key Expansion

The key expansion process in AES generates a series of round keys from the initial cipher key. Each round key is used in the *AddRoundKey* step of AES.

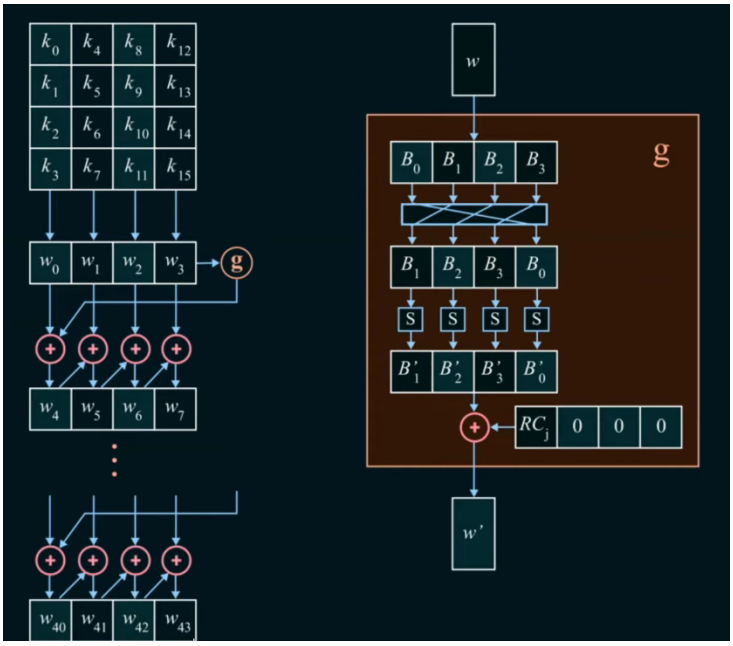


Figure 5: AES key expansion process. The original key is split into words w_0 to w_3, then a recursive process using the function g, S-box substitution, and round constants RC_j generates the remaining words for all round keys.

Brief explanation:

- The original key is divided into 4 words (w_0 to w_3 for AES-128).
- Each new word w_i is generated using a combination of the previous word and a transformation function g.
- The function g includes a rotation, substitution through the S-box, and addition of a round constant RC_j.
- This process continues until all round keys (w_0 to w_{43} for AES-128) are generated.

1.5 AES ModeLS

1.5.1 ECB Mode

Overview

AES in **ECB (Electronic Codebook) mode** encrypts each block of plaintext independently using the same key. It is the simplest mode of operation but has some important limitations.

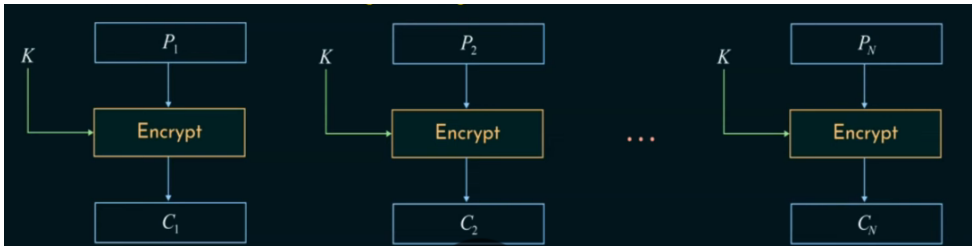


Figure 6: AES encryption using ECB mode

Brief Explanation

In ECB mode, the plaintext is divided into 128-bit blocks. Each block is encrypted separately with AES. Identical plaintext blocks produce identical ciphertext blocks, which can reveal patterns in the data. ECB is easy to implement and suitable for small amounts of data without repeated blocks.

Advantages

- Simple and easy to implement.
- Parallel encryption of blocks is possible.
- Useful for encrypting small, independent blocks of data.

Limitations

- Reveals patterns when plaintext blocks repeat.
- Not recommended for encrypting large or structured data.
- Less secure compared to other modes like CBC or GCM.

1.5.2 CBC Mode

Overview

AES in **CBC (Cipher Block Chaining) mode** encrypts each block of plaintext by combining it with the previous ciphertext block before applying AES. An **Initialization Vector (IV)** is used for the first block to ensure uniqueness.

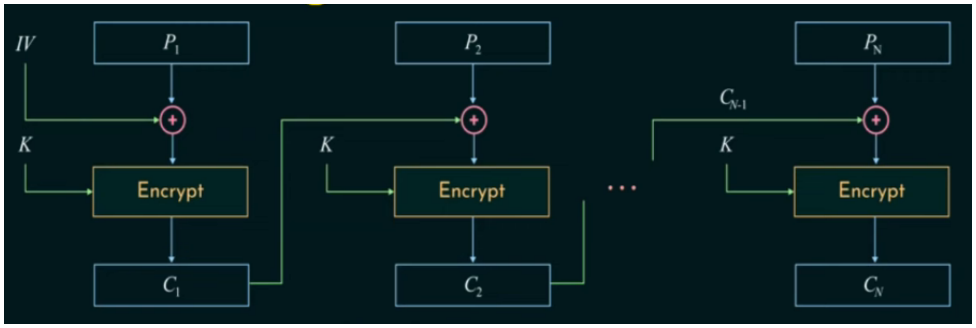


Figure 7: AES encryption using CBC mode

Brief Explanation

In CBC mode, each plaintext block is XORed with the previous ciphertext block before encryption. This ensures that identical plaintext blocks produce different ciphertext blocks, preventing patterns from being visible. The first block uses a random IV to guarantee uniqueness.

Advantages

- Hides patterns in plaintext, unlike ECB.
- More secure for large or structured data.
- Commonly used and well-supported in cryptographic libraries.

Limitations

- Encryption cannot be fully parallelized (each block depends on the previous one).
- Decryption can be parallelized, but encryption latency is higher.
- Requires a unique and random IV for each message.

1.5.3 CFB Mode

Overview

AES in **CFB (Cipher Feedback) mode** converts a block cipher into a self-synchronizing stream cipher. It encrypts small units of plaintext (usually 8 or 128 bits) and feeds the ciphertext back into the encryption process.

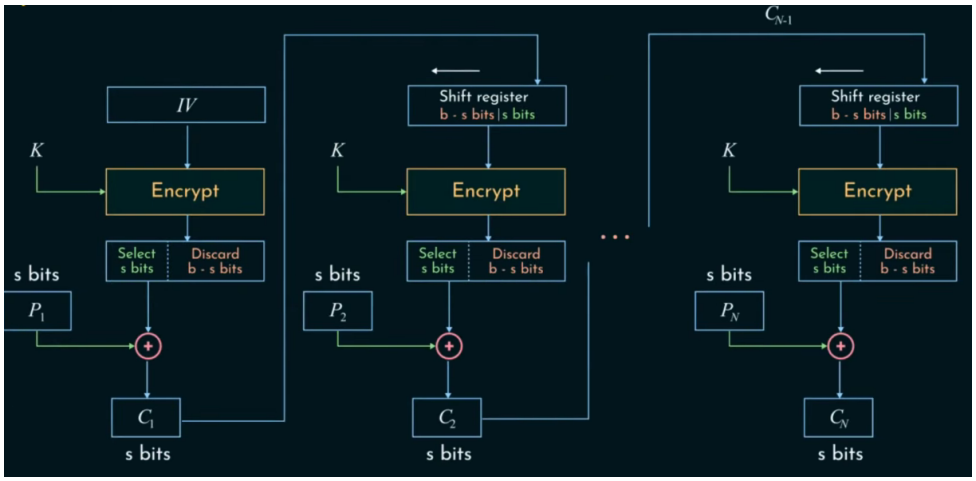


Figure 8: AES encryption using CFB mode

Brief Explanation

In CFB mode, the previous ciphertext block is encrypted and then XORed with the current plaintext block to produce the ciphertext. This makes CFB a stream-like mode that can encrypt data smaller than the AES block size and allows encryption of data in real-time.

Advantages

- Can encrypt data smaller than the block size.
- Self-synchronizing: a lost block does not disrupt the entire stream.
- Suitable for real-time applications.

Limitations

- Encryption cannot be fully parallelized.
- Slightly slower than ECB or CBC due to feedback processing.
- Requires careful handling of the IV to maintain security.

1.5.4 CTR Mode

Overview

AES in **CTR (Counter) mode** transforms AES into a stream cipher. Instead of encrypting plaintext blocks directly, a counter value is encrypted and then XORed with the plaintext block to produce the ciphertext.

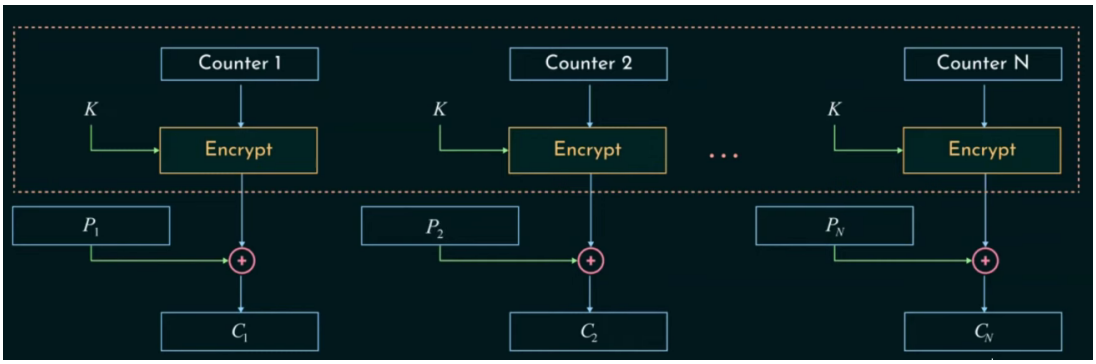


Figure 9: AES encryption using CTR mode

Brief Explanation

In CTR mode, a counter (nonce + increment) is encrypted with AES, producing a keystream block. Each plaintext block is XORed with the corresponding keystream block to generate the ciphertext. CTR allows parallel encryption and decryption, and avoids revealing patterns in repeated plaintext blocks.

Advantages

- Parallelizable, which speeds up encryption and decryption.
- Patterns in plaintext are not exposed.
- Can encrypt data of arbitrary length without padding.

Limitations

- Requires a unique counter value for each block to ensure security.
- Mismanagement of nonce/counter can compromise security.
- Slightly more complex than ECB or CBC modes.

2 Asymmetric Cryptography

Asymmetric cryptography uses a **pair of keys**:

- **Public Key:** shared with everyone.
- **Private Key:** kept secret.

Data encrypted with one key can only be decrypted with the other. This ensures confidentiality, authentication, and secure key exchange.

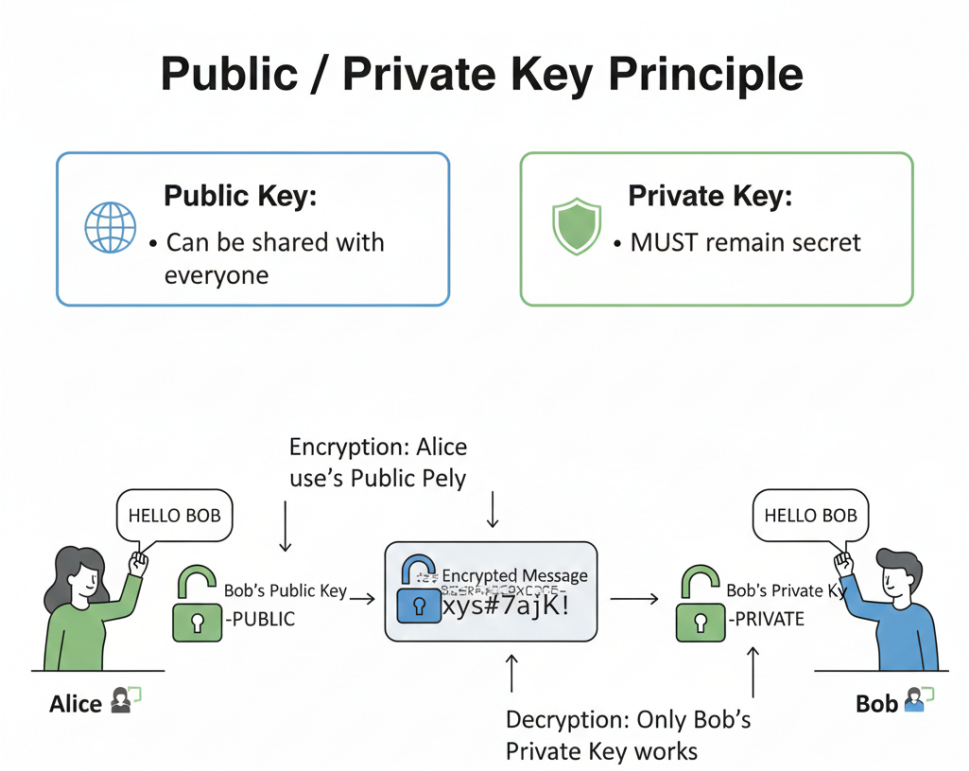


Figure 10: How Public and Private Keys Work

2.1 One-way Functions

Security relies on **one-way functions**:

- Easy to compute in one direction.
- Very hard to reverse.

Key points:

- Given an input x, it is easy to compute $f(x)$.
- Given $f(x)$, it is very hard to find the original input x.

Examples:

- Multiplying large numbers — easy
- Factoring large numbers — very hard
- Modular exponentiation — easy
- Discrete logarithm — very hard

These functions are the foundation of RSA, DH, and ElGamal.

2.2 Core Systems

2.2.1 RSA

Principle: based on the difficulty of factoring large numbers.
How it works: encrypt with public key, decrypt with private key.
Key Generation:

1. Choose two large primes p and q, compute $n = p \times q$
2. Compute $\phi(n) = (p-1)(q-1)$
3. Choose e where $\gcd(e, \phi(n)) = 1$
4. Calculate d where $d \times e \equiv 1 \pmod{\phi(n)}$
5. **Public Key:** (n, e) **Private Key:** (n, d)

Encryption/Decryption:

- **Encrypt:** $C = M^e \mod n$
- **Decrypt:** $M = C^d \mod n$

Limitations:

- Minimum 2048-bit keys required
- Vulnerable to factoring attacks and quantum computers (Shor's algorithm)
- Must use padding (OAEP/PSS) to prevent attacks
- Much slower than symmetric encryption (100-1000x)

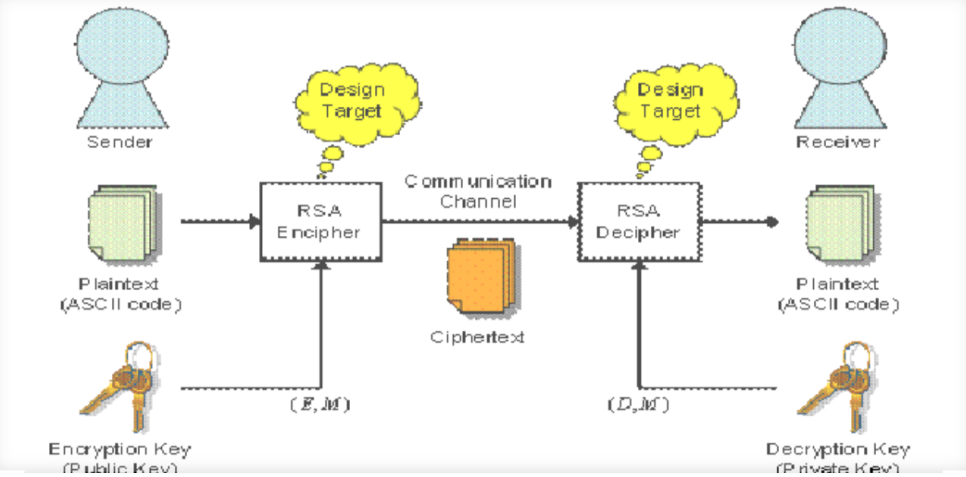


Figure 11: RSA Encryption / Decryption

Applications: secure email, digital signatures, authentication.

2.2.2 Diffie-Hellman (DH)

Principle: based on the discrete logarithm problem.
Purpose: securely share a secret key over an insecure channel.
Key Exchange Process:

1. **Public parameters:** prime p and generator g
2. **Alice:** chooses secret a, computes $A = g^a \mod p$, sends A
3. **Bob:** chooses secret b, computes $B = g^b \mod p$, sends B
4. **Shared secret:** Both compute $K = g^{ab} \mod p$
    - Alice: $K = B^a \mod p$
    - Bob: $K = A^b \mod p$

Limitations:

- **No authentication:** vulnerable to Man-in-the-Middle (MITM) attacks
- Must be combined with signatures or certificates for security
- Only establishes shared key, does not encrypt data
- **Modern variants:** ECDH (smaller keys), DHE (perfect forward secrecy)

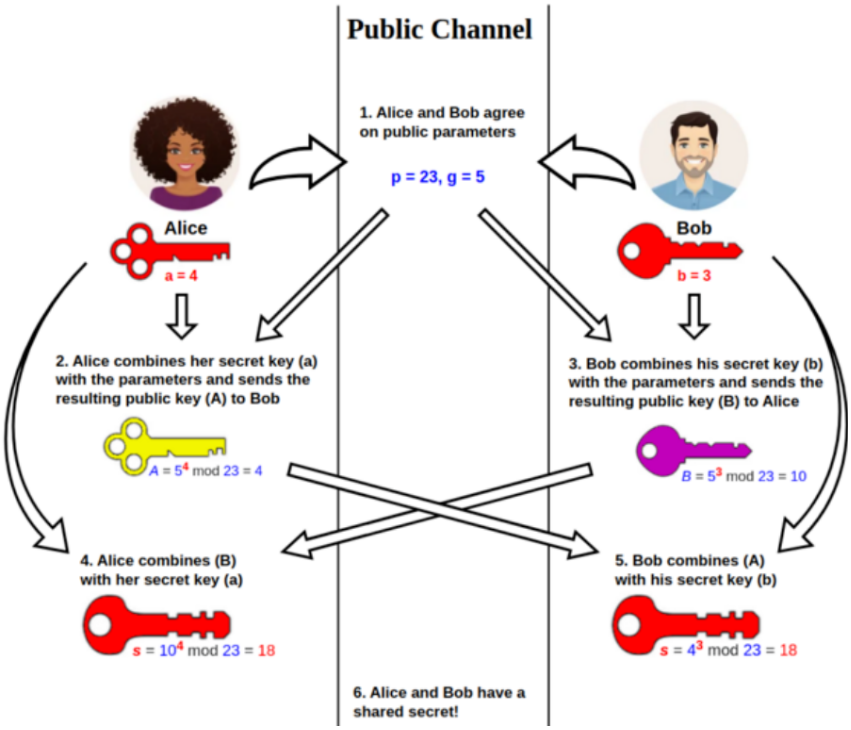


Figure 12: Diffie-Hellman Key Exchange

Applications: creating session keys for TLS/SSL, VPNs.

2.2.3 ElGamal

Principle: based on discrete logarithms.
How it works: probabilistic encryption using recipient's public key.
Key Generation:

1. Choose prime p and generator g
2. Choose private key x, compute public key $y = g^x \mod p$
3. **Public Key:** (p, g, y) **Private Key:** x

Encryption/Decryption:

- **Encrypt:** Choose random k, compute:
  - $c_1 = g^k \mod p$
  - $c_2 = M \times y^k \mod p$
- **Decrypt:** Compute $s = c_1^x \mod p$, then $M = c_2 \times s^{-1} \mod p$

Limitations:

- **Ciphertext expansion:** doubles message size (two components)
- Must use fresh random k for each message (reusing k breaks security)
- Based on discrete logarithm problem (similar security to DH)
- **Advantage:** Probabilistic encryption (same message $\rightarrow$ different ciphertexts)

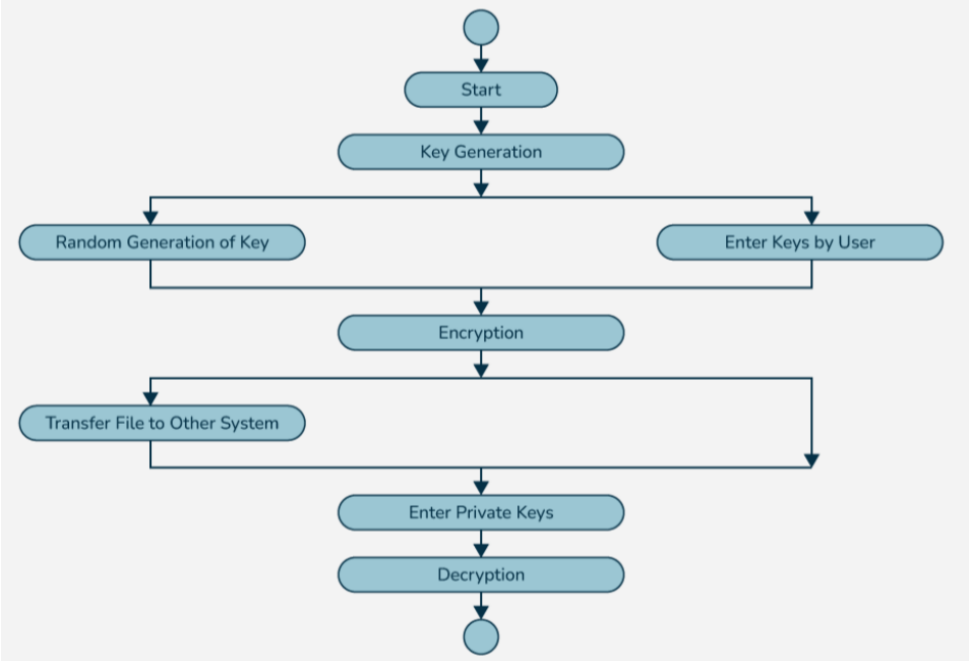


Figure 13: ElGamal Encryption / Decryption

Applications: secure messaging, encryption systems, digital signatures.

2.3 Roles of Asymmetric Cryptography

Asymmetric cryptography serves multiple purposes:

- **Encryption:** protect data with the recipient's public key.
- **Digital Signature:** sign with private key, verify with public key.
- **Key Exchange:** establish shared secrets for symmetric encryption (e.g., AES).

2.4 Summary / Key Points

- RSA: secure and widely used for encryption and digital signatures.
- Diffie-Hellman: key exchange to establish shared secrets.
- ElGamal: probabilistic encryption for added security.
- One-way functions: fundamental to asymmetric cryptography.
- Roles: encryption, authentication, key exchange.

2.5 Real-world Applications (brief)

- **SSH:** secure remote login.
- **TLS/SSL:** secure web connections.
- **VPNs:** secure network connections.
- **Authentication systems:** passwordless login, digital signatures.

3 Resources

3.1 Video Tutorials

- **AES Explained:**
  `https://youtu.be/3MPkc-PFSRI`
- **RSA & ElGamal Cryptography:**
  `https://youtu.be/DjsaVpE_kdw`
- **Diffie-Hellman Key Exchange:**
  `https://youtu.be/d1KXDGgwIpA`
- **One-Way Functions:**
  `https://youtu.be/GE3dsA5S7M8`